



Department of Electronic & Telecommunication Engineering
University of Moratuwa, Sri Lanka

BM4152 Biosignal Processing

Assignment 01

Nadha M.I.
220409J

Date: 12.09.2025

1. Smoothing Filters

1.1. Moving Average MA(N) Filter

1.1.1 Preliminaries

The `ECG_template` signal (Figure 1) shows a clear, low-noise single PQRST complex sampled at $f_s = 500$ Hz with a small P wave bump near 0.12 s, a sharp QRS complex peaking at approximately 0.75 mV near 0.21 s, and a broader T wave near 0.44 s. Zero-mean Gaussian white noise scaled to a target SNR of 5 dB was added to produce `nECG` (Figure 2). At this SNR the large-amplitude R-wave remains just visible above the noise, but the smaller P-wave and T-wave are almost entirely obscured.

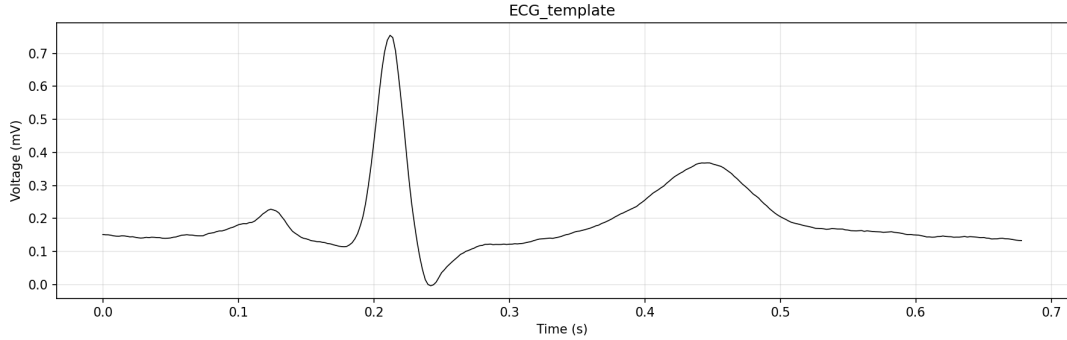


Figure 1: `ECG_template` signal.

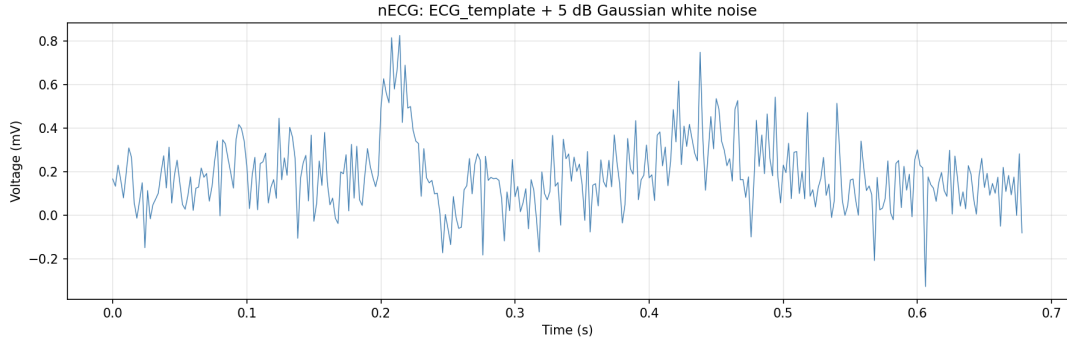


Figure 2: `nECG`: `ECG_template` with 5 dB Gaussian white noise added.

The PSD of `nECG` (Figure 3) shows that most of the signal power is concentrated below approximately 20 Hz, where the main ECG components are present. At higher frequencies, the added white noise becomes more dominant.

1.1.2 MA(3) Filter: Custom Implementation

A custom function was used to implement the MA(3) filter directly using Equation (1):

$$y[n] = \frac{1}{3} \sum_{k=0}^2 x[n-k] = \frac{x[n] + x[n-1] + x[n-2]}{3}.$$

The MA(3) filter has a group delay of

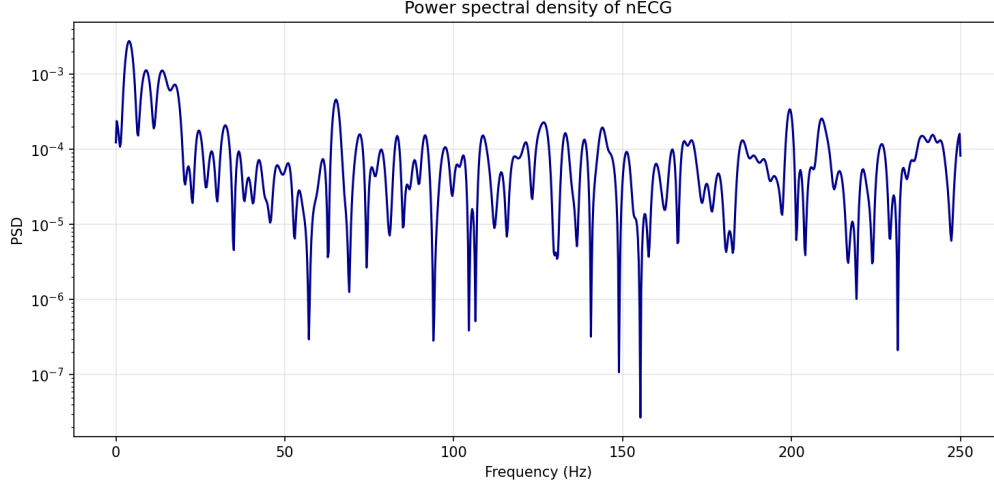


Figure 3: Power spectral density estimate of nECG.

$$D = \frac{N - 1}{2} = \frac{3 - 1}{2} = 1 \text{ sample},$$

which corresponds to 2 ms at $f_s = 500$ Hz. After compensating for this delay, `ma3ECG_1` follows the main shape of the `ECG_template`. The P-wave and T-wave are preserved, while the QRS peak is slightly reduced and smoothed. Some noise is still present because the 3-sample window provides only limited smoothing.

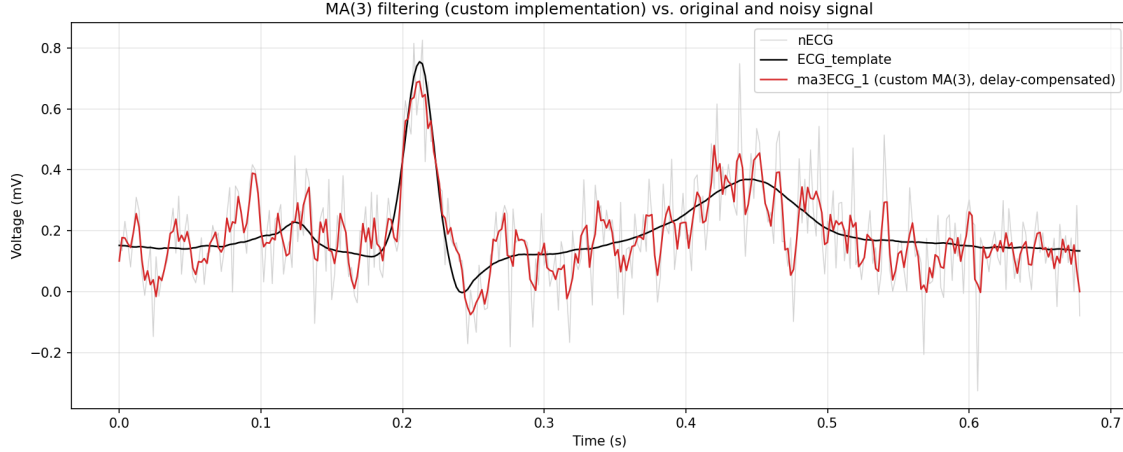


Figure 4: Delay-compensated `ma3ECG_1` (custom MA(3) implementation) vs. `ECG_template` and `nECG`.

The overlapping PSDs in Figure 5 show the expected lowpass behaviour of the MA(3) filter. Below approximately 50 Hz, the filtered PSD is close to the noisy PSD, so the main ECG components are mostly preserved. Above approximately 50–60 Hz, the filtered PSD is lower than the noisy PSD, showing that the MA(3) filter reduces high-frequency noise.

1.1.3 MA(3) Filter: Built-in Function

The same MA(3) filter was implemented using `scipy.signal.lfilter()`. As expected, the resulting waveform (Figure 6) is visually identical to the custom implementation in

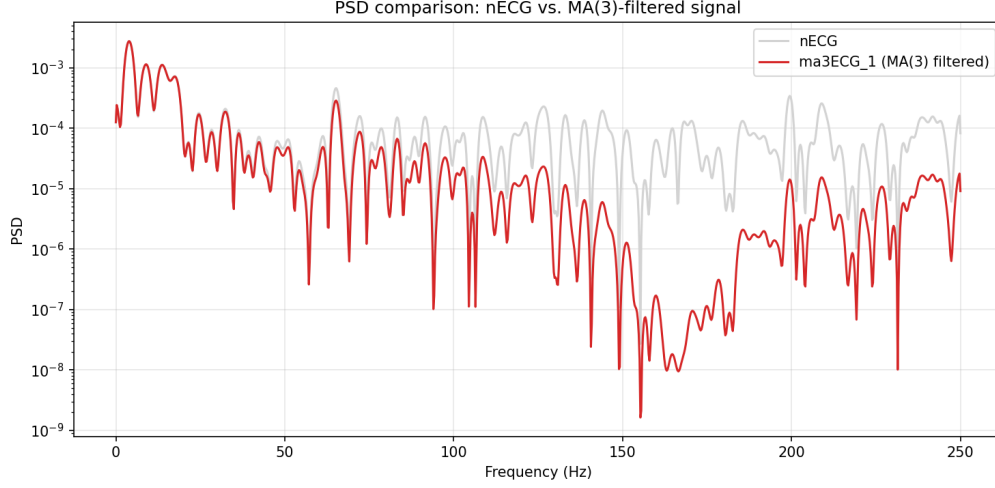


Figure 5: PSD comparison: nECG vs. ma3ECG_1 (MA(3) filtered).

Figure 4, confirming that the two implementations are equivalent.

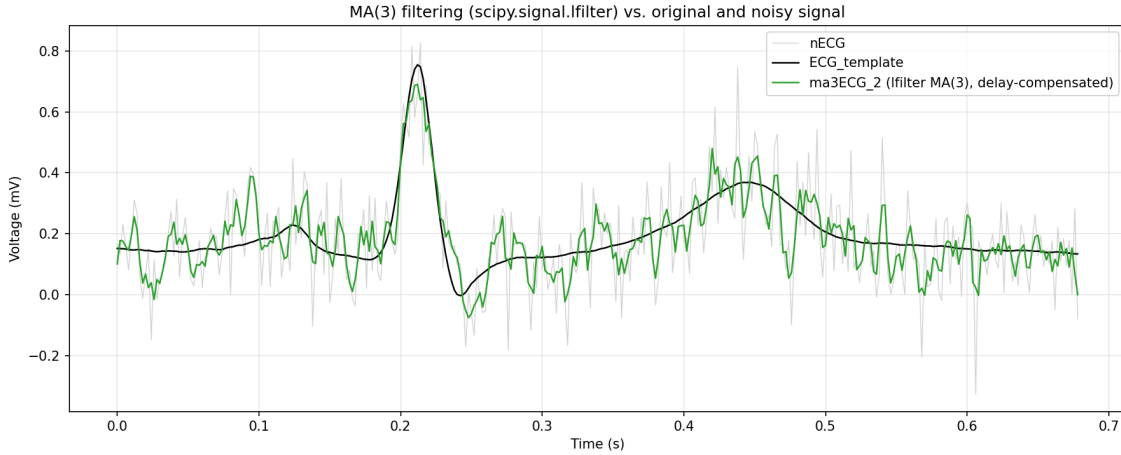


Figure 6: Delay-compensated ma3ECG_2 (`lfilter` MA(3)) vs. ECG_template and nECG.

Figure 7 shows the MA(3) filter's magnitude response, phase response, and pole-zero plot. The magnitude response is a smooth low-pass shape with a null at $f_s/3 \approx 167$ Hz. The phase response is approximately linear, with sudden jumps at frequencies where the magnitude response becomes zero, and the pole-zero plot shows two zeros symmetrically placed on the unit circle at $\pm 2\pi/3$ rad, with both poles located at the origin.

1.1.4 MA(10) Filter

Figure 8 compares the MA(3) and MA(10) filters. The MA(10) filter has a narrower main lobe, and has more ripples in the passband. Its zeros produce nulls at multiples of $f_s/10 = 50$ Hz.

The pole-zero plot shows that MA(10) has nine zeros evenly distributed around the unit circle, while MA(3) has only two zeros. The longer MA(10) filter also has a larger group delay:

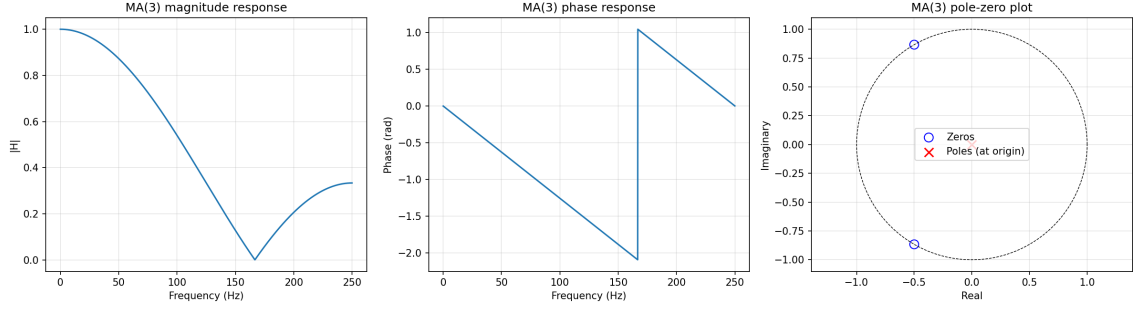


Figure 7: MA(3) filter: magnitude response, phase response, and pole-zero plot.

$$D = \frac{N - 1}{2} = \frac{10 - 1}{2} = 4.5 \text{ samples,}$$

compared with 1 sample for MA(3).

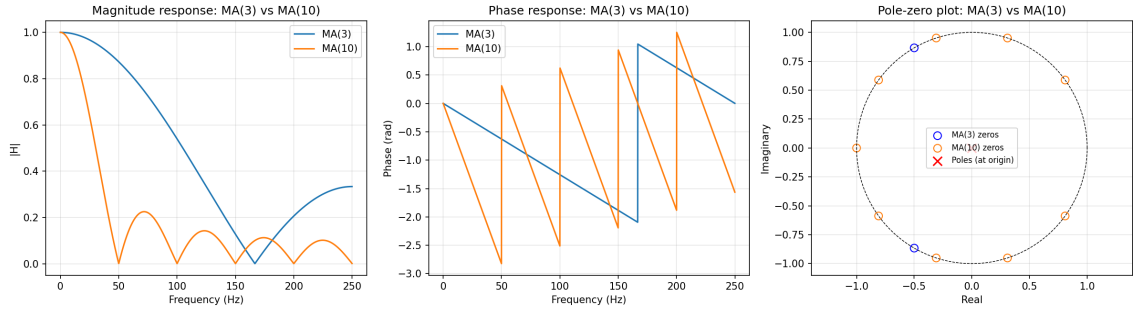


Figure 8: Magnitude response, phase response, and pole-zero plot: MA(3) vs. MA(10).

The time-domain comparison in Figure 9 shows that `ma10ECG` is smoother than `ma3ECG_2`, with less high-frequency noise. However, the QRS peak is also more reduced and rounded. This shows the trade-off between better noise reduction and preserving the original ECG shape when using a longer averaging window.

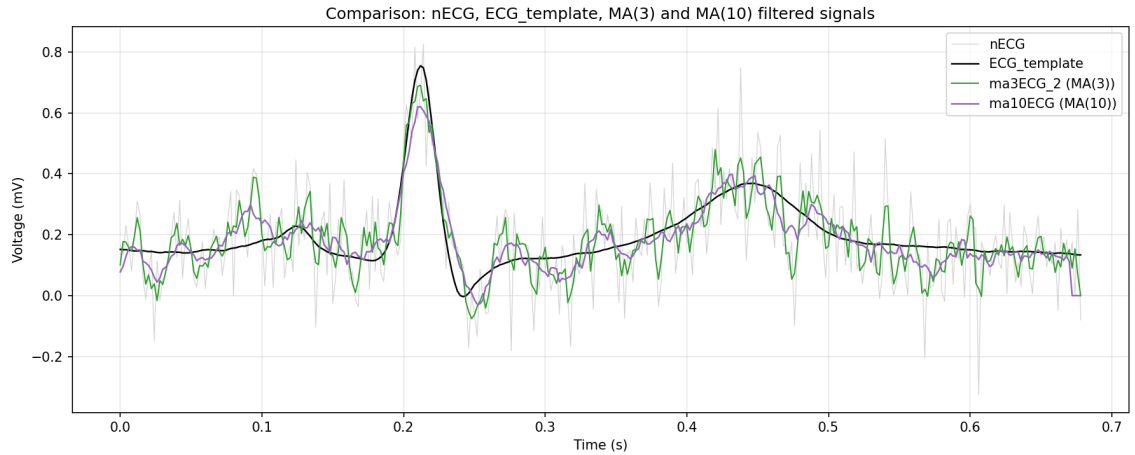


Figure 9: Comparison of `nECG`, `ECG_template`, `ma3ECG_2` (MA(3)), and `ma10ECG` (MA(10)).

1.1.5 Optimum MA(N) Filter Order

The mean-squared-error (MSE) was used to measure the difference between the noise-free ECG signal and the filtered signal. It was calculated using

$$\text{MSE} = \frac{1}{M} \sum_{n=1}^M (x[n] - \hat{x}[n])^2,$$

where $x[n]$ is the noise-free `ECG_template`, $\hat{x}[n]$ is the delay-compensated output of the MA(N) filter, and M is the number of samples being compared.

The MA(N) filter was tested for $N = 1$ to $N = 100$. As shown in Figure 10, the lowest MSE was obtained at $N = 12$, with an MSE of 0.0023. The MSE decreases quickly for small values of N and then gradually increases after $N = 12$.

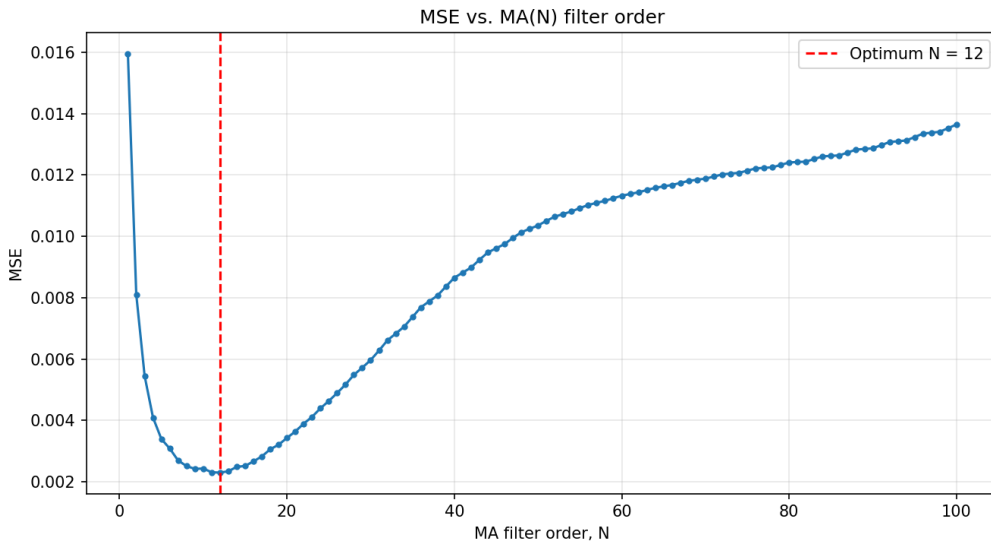


Figure 10: MSE vs. MA(N) filter order.

Reasons for large MSE values at low and high filter orders . At low filter orders, the averaging window is short, so there is less noise reduction and much of the white noise remains in the filtered ECG. Therefore, the filtered signal is more different from the noise-free `ECG_template`, resulting in a large MSE. As the filter order increases, more noise is removed and the MSE decreases. However, at very high filter orders, the averaging window becomes too wide and starts to smooth important changes in the ECG waveform. This causes distortion of the original signal, so the MSE increases again.

1.2. Savitzky-Golay SG(N,L) Filter

1.2.1 Application of the SG(3,11) Filter

From the figure11, it can be observed that the SG(3,11) filter smooths the noisy signal while keeping the main shape of the ECG visible. The QRS complex is still clearly present, but some small fluctuations remain in the filtered signal. Larger variations can also be observed near the beginning and end of the signal, where the waveform is expected to be relatively flat.

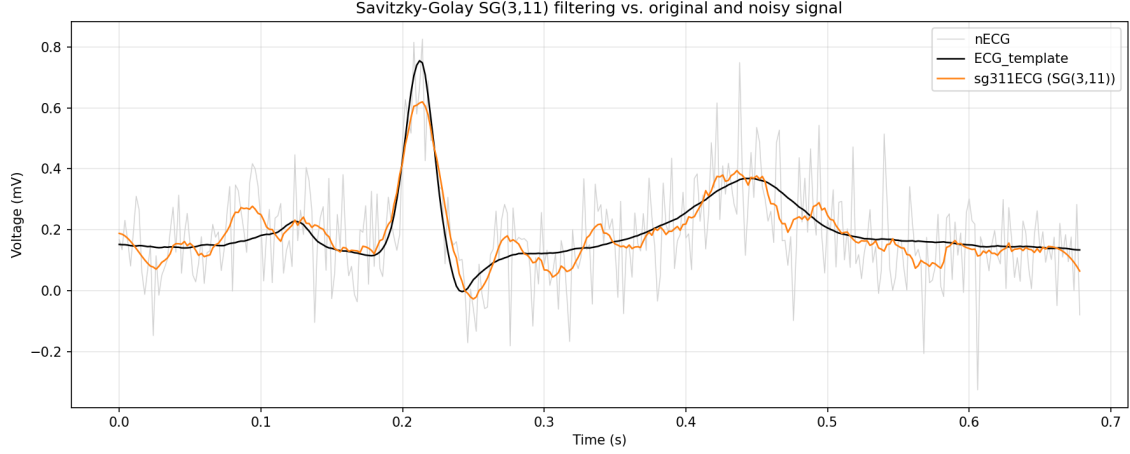


Figure 11: Savitzky-Golay SG(3,11) filtered signal (`sg311ECG`) vs. `ECG_template` and `nECG`.

1.2.2 Optimum SG(N,L) Filter Parameters

The polynomial order N (2 to 7) and half-window parameter L (2 to 25) were swept over a grid, computing the MSE against `ECG_template` for every valid combination (invalid combinations with $N > L' - 1$ were excluded). The resulting MSE surface (Figure 12) identifies the optimum parameters as $N = 2$, $L = 13$ (MSE = 0.001941).

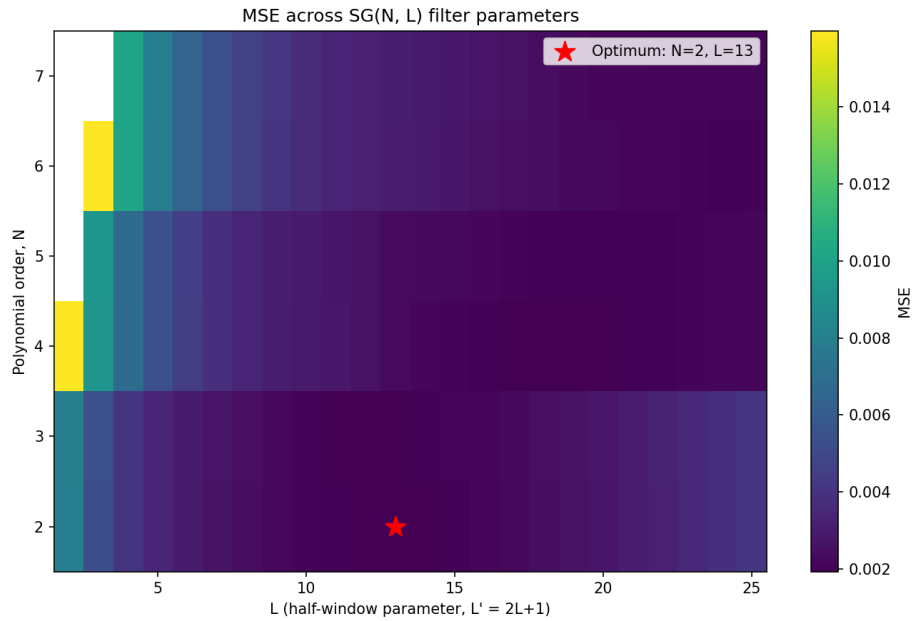


Figure 12: MSE across SG(N, L) filter parameters (pcolormesh view).

From Figure 13, it can be observed that the optimum SG(2,13) filter produces a smoother signal with less high-frequency noise than `sg311ECG`. However, the wider window also makes the QRS peak more rounded and slightly reduces its amplitude.

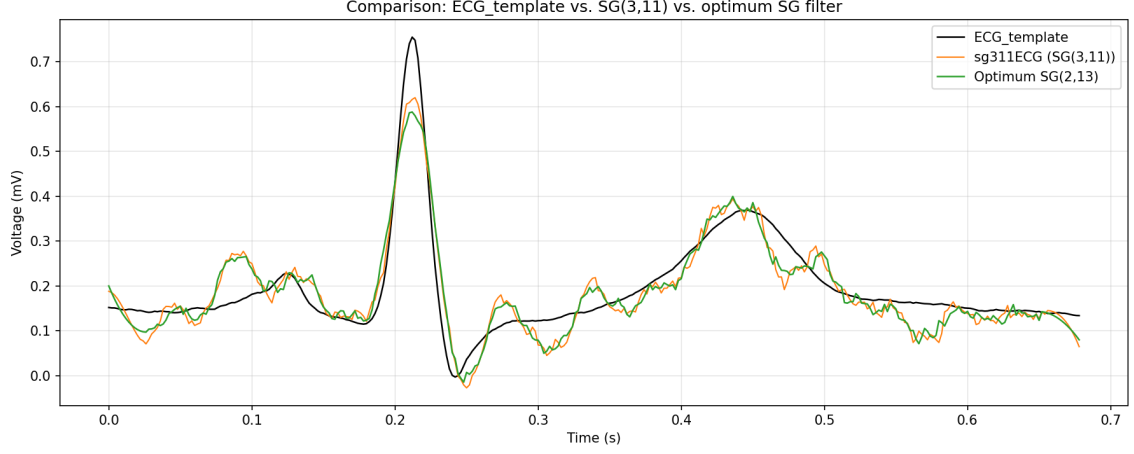


Figure 13: Comparison of ECG_template, sg311ECG (SG(3,11)), and the optimum SG(2,13) filter.

1.2.3 Comparison of Optimum MA(N) and SG(N,L) Filters

The SG(2,13) filter gives a slightly lower MSE than the MA(12) filter. This is because the SG filter fits a polynomial to the samples within the window instead of simply taking their average. Therefore, it can preserve changes in the ECG waveform more effectively while still reducing noise. From the figure, both filtered signals have a similar overall shape, although the SG filter gives slightly smoother results. The downward change at the end of the MA(12) signal is caused by the delay compensation used for the causal MA filter and is not part of the actual filtered ECG signal.

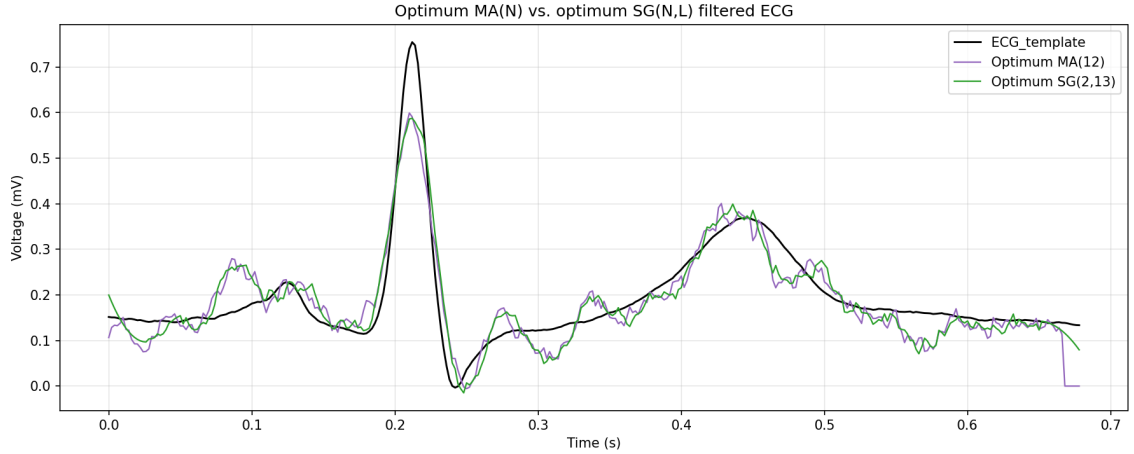


Figure 14: Optimum MA(12) vs. optimum SG(2,13) filtered ECG.

In terms of computational complexity, the MA(N) filter is simpler than the SG(N,L) filter. The MA filter mainly requires additions and a division for each output sample, while the SG filter uses a set of pre-computed coefficients to perform a longer FIR filtering operation.

Therefore, the SG filter provides a slightly lower MSE, but the MA filter is simpler and computationally cheaper.

2. Ensemble Averaging

2.1. Signal with Multiple Measurements (ABR)

2.1.1 Preliminaries

Figure 15 shows the raw stimulus train together with the corresponding ABR+EEG recording. The stimulus train consists of repeated stimulus pulses occurring at regular intervals. The ABR recording is a much smaller-amplitude waveform compared with the stimulus. The ABR waveform is time-locked to the stimulus, meaning that similar ABR responses occur following each stimulus.

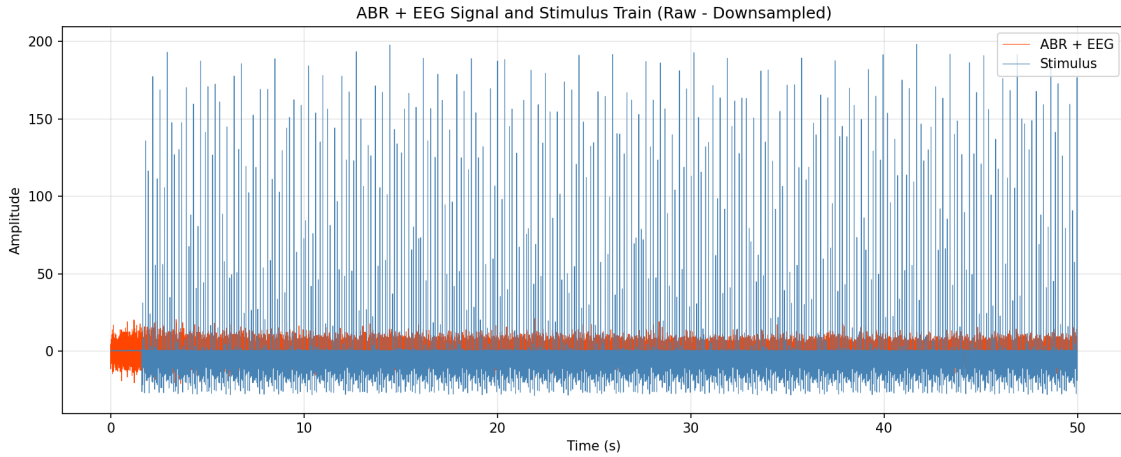


Figure 15: Raw stimulus train and ABR+EEG recording over the full acquisition period.

A voltage threshold of 50 was used to detect the stimulus onsets. A total of 1033 stimuli were detected, and an epoch from 2 ms before to 10 ms after each onset was extracted. The 1033 ABR epochs were then ensemble averaged. This preserves the time-locked ABR response while reducing random background noise. The averaged waveform clearly shows the characteristic ABR morphology, including waves I–VI, as shown in Figure 16.

2.1.2 Improvement of the SNR

Figure 17 shows the MSE, MSE_k , between the final ensemble-averaged ABR and the average of the first k epochs, where k ranges from 1 to 1033. The MSE decreases rapidly as more epochs are averaged, especially during the first approximately 100 epochs. After this, the decrease becomes more gradual.

This is because ensemble averaging preserves the repeated ABR response while reducing random noise. The noise amplitude theoretically decreases by a factor of $1/\sqrt{k}$ when k independent epochs are averaged. Therefore, the SNR improves as more epochs are included, but the improvement becomes smaller with additional epochs. At $k = 1033$, the progressive average is the final ensemble average, so the MSE approaches zero.

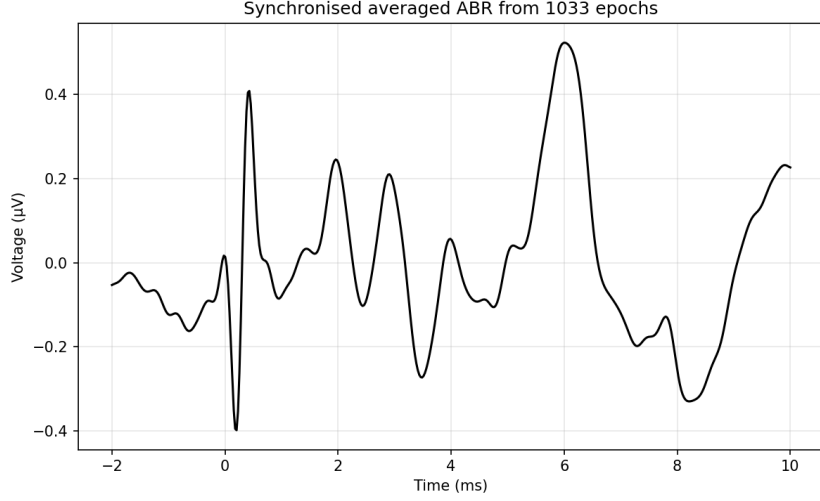


Figure 16: Ensemble-averaged ABR waveform from 1033 epochs, showing resolvable waves I–VI.

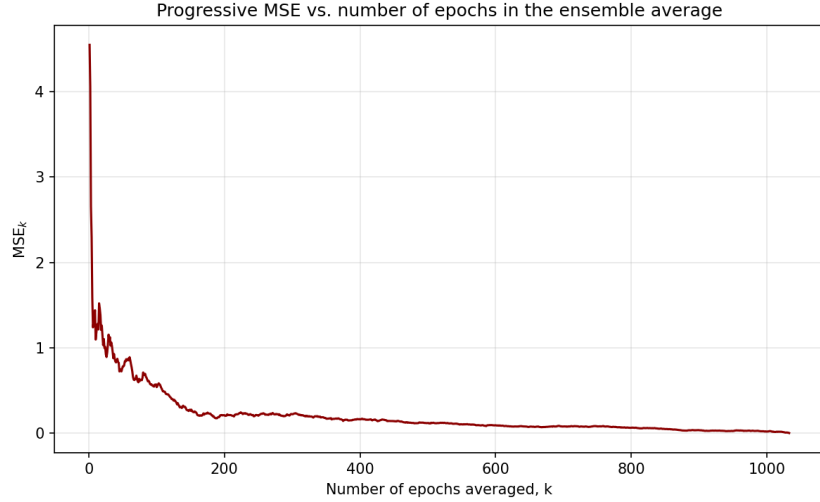


Figure 17: Progressive MSE, MSE_k , versus the number of ABR epochs k included in the running average.

2.2. Signal with Repetitive Patterns (ECG)

2.2.1 Viewing the Signal and Addition of Gaussian White Noise

The raw recording (Figure 19) exhibits clearly defined, regularly-spaced QRS complexes with a consistent morphology, indicating a stable underlying rhythm. A single representative PQRST waveform was extracted and stored as `ECG_template` (Figure 20), showing well-defined P, Q, R, S and T waves.

Zero-mean Gaussian white noise scaled to a target SNR of 5 dB was added to the ECG recording to produce `nECG` (Figure 21).

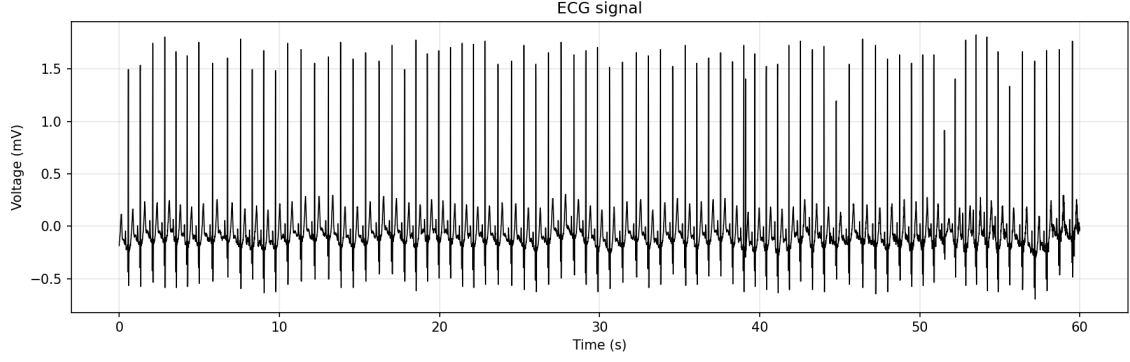


Figure 18: Raw ECG recording (Lead II).

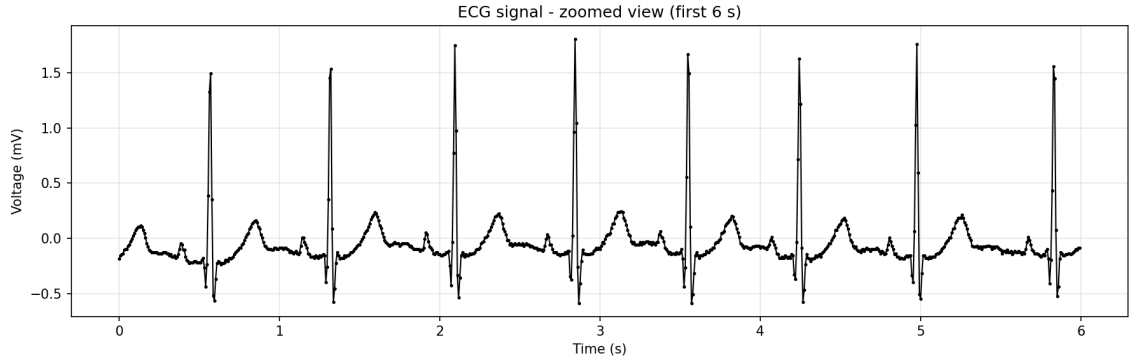


Figure 19: Raw ECG recording (Zoomed in version).

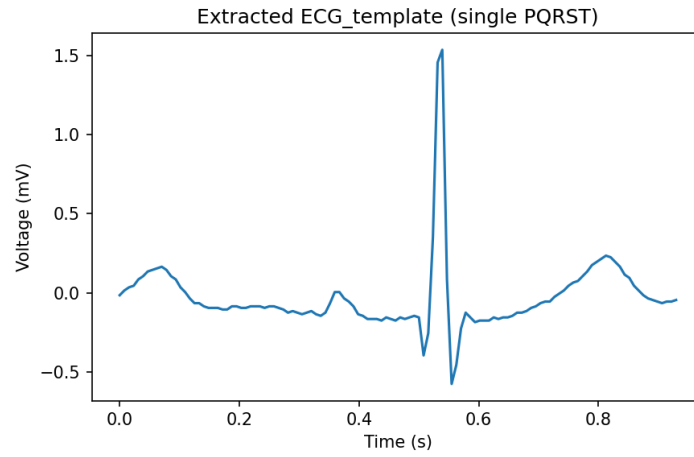


Figure 20: Extracted single PQRST waveform used as `ECG_template`.

2.2.2 Segmenting ECG into Separate Epochs and Ensemble Averaging

The normalised cross-correlation between `ECG_template` and `nECG` (Figure 22) shows a series of sharp and regularly-spaced peaks, corresponding to the individual heartbeats. The peaks are clearly distinguishable from the surrounding noise.

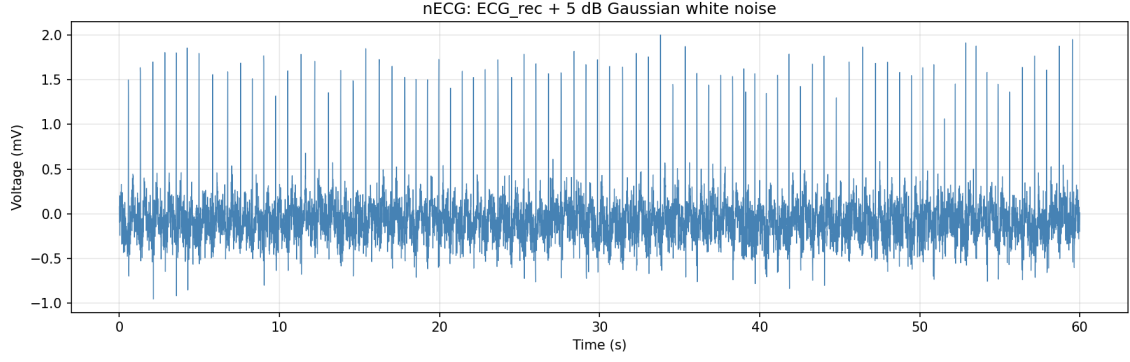


Figure 21: Noisy ECG signal (**nECG**) after adding 5 dB Gaussian white noise.

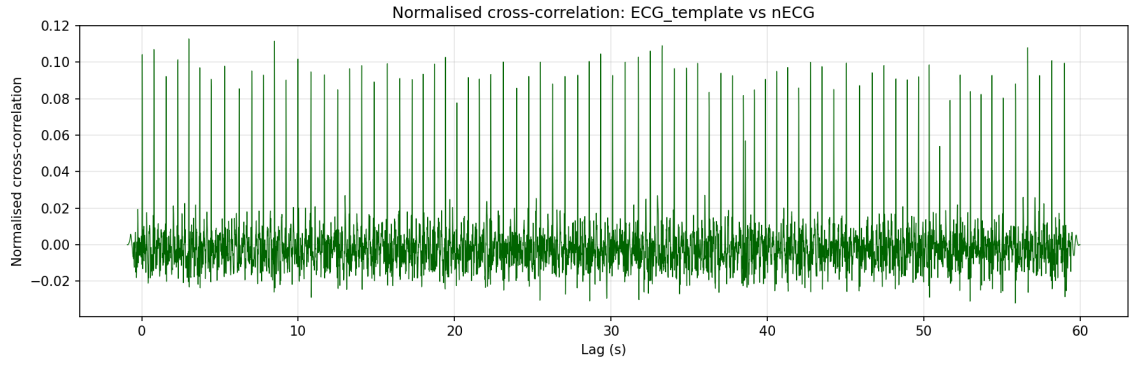


Figure 22: Normalised cross-correlation between **ECG_template** and **nECG**.

A relative threshold of 40% of the maximum correlation value (≈ 0.045) was therefore used (via `scipy.signal.find_peaks`) rather than a fixed absolute threshold, since the absolute correlation magnitude is data- and noise-dependent. This detected 79 ECG pulses over the 60-second recording.

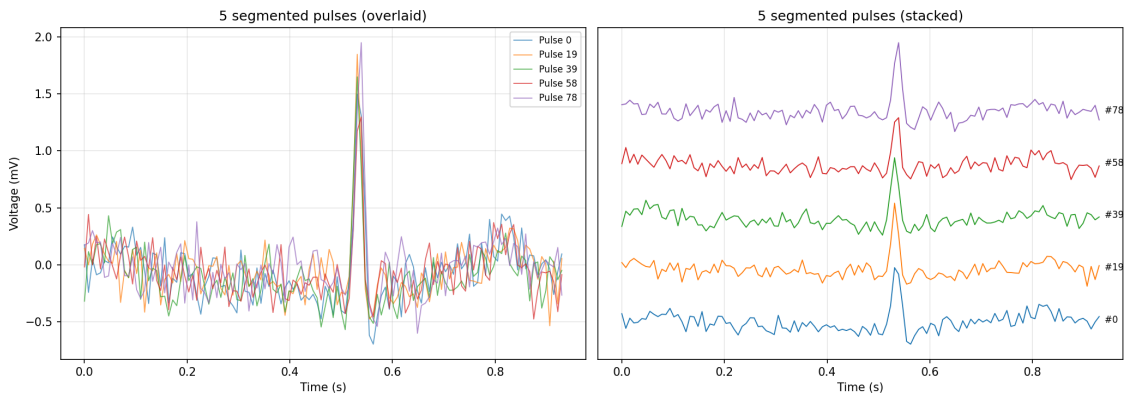


Figure 23: Five representative segmented ECG pulses shown overlaid (left) and stacked (right), sampled evenly across the recording.

Figure 23 shows that cross-correlation-based segmentation correctly aligns the dominant R-wave across epochs, with only minor residual jitter.

The progressive MSE for the ECG ensemble average (Figure 24) shows a similar trend

to the ABR. The MSE decreases quickly during the first ~ 10 pulses, from approximately 0.16 to 0.04. After this, the MSE decreases more slowly as more pulses are added. This shows that ensemble averaging reduces the noise more effectively when the number of averaged pulses is increased, following the expected $1/\sqrt{k}$ noise-suppression behaviour.

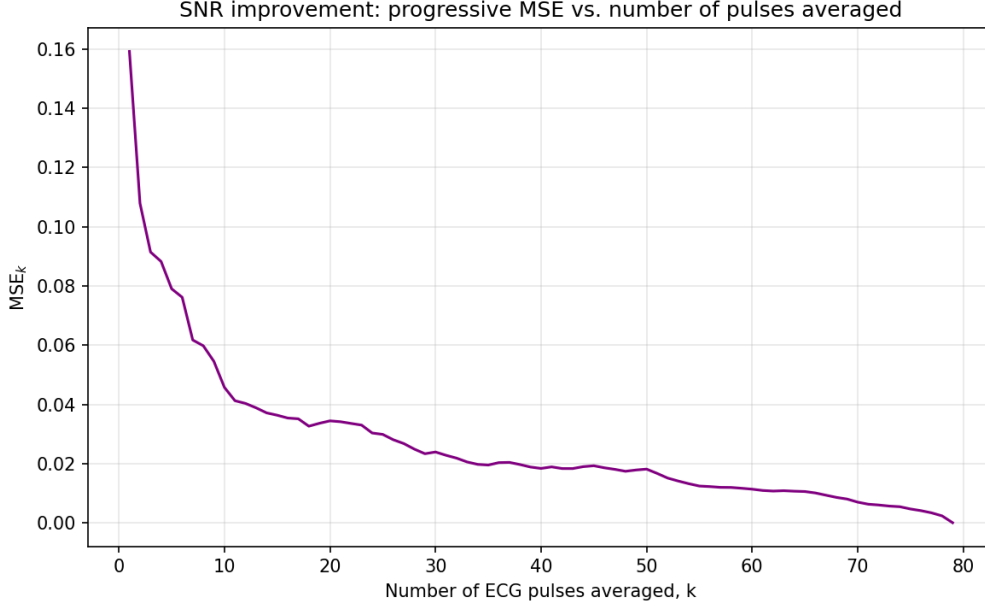


Figure 24: Progressive MSE, MSE_k , versus the number of ECG pulses k included in the running ensemble average.

Figure 25 compares a single noisy ECG pulse with ensemble-averaged pulses using 10 pulses and all 79 segmented pulses. The single noisy pulse contains noticeable random noise, especially in the low-amplitude regions. Averaging 10 pulses reduces the noise and makes the ECG waveform clearer. Averaging all 79 pulses provides further noise reduction and produces a smoother waveform. The R-wave remains clearly visible in the noisy pulse because its amplitude is much larger than the noise.

Justification: Correlation-based segmentation vs R-wave detection

Using the maximum cross-correlation with the full PQRST template is preferable to using only R-wave detection. Cross-correlation considers the shape of the entire ECG waveform, including the P, Q, R, S and T waves, rather than relying on only the R-wave which helps to align the ECG pulses more accurately.

It is also more robust to noise. Noise peaks can be incorrectly detected as R-waves, while a true R-wave may be reduced by noise. Cross-correlation compares the complete waveform with the template, so a true ECG pulse can still produce a strong correlation even when some parts are affected by noise. Therefore, correlation-based segmentation provides more reliable pulse detection and alignment, resulting in a clearer ensemble-averaged waveform.

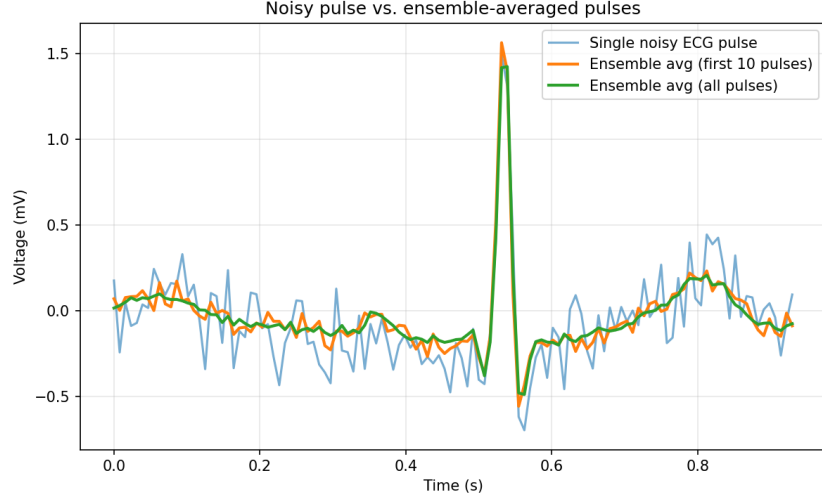


Figure 25: Comparison of a single noisy ECG pulse against ensemble averages of 10 and 79 pulses.

3. Designing FIR Filters Using the Windowing Method

3.1. Characteristics of Window Functions

3.1.1 Rectangular Window: Effect of Length M

Figure 26 shows the impulse responses of FIR filters designed using a rectangular window for $M = 5, 50, 100$. As M increases, the impulse response becomes longer because the filter has more coefficients. The response for $M = 5$ is short, while $M = 50$ and $M = 100$ have increasingly longer responses. Therefore, increasing the window length increases the length of the FIR filter and allows a narrower transition band, but also increases the number of coefficients and the computational cost.

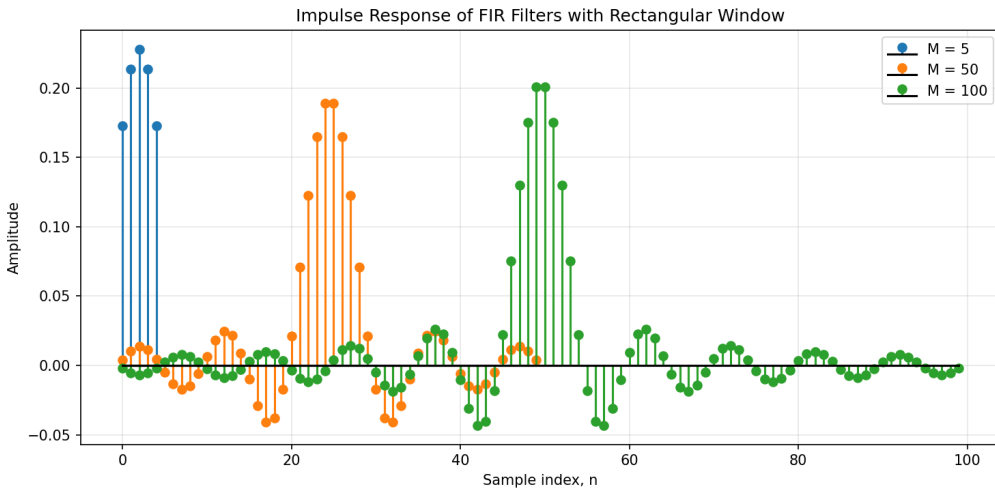


Figure 26: Rectangular window impulse response, overlaid for $M = 5, 50, 100$.

The corresponding lowpass filter response ($\omega_c = 0.4\pi$) is shown in Figure 27. As M increases from 5 to 50 and 100, the transition band becomes narrower and the filter

becomes sharper. Therefore, a larger window length gives better frequency selectivity. However, increasing M does not significantly reduce the ripple near the cutoff frequency. Instead, the ripples become narrower and more closely spaced. This ripple is caused by the abrupt edges of the rectangular window.

The phase response is approximately linear for all three filters. This means that the different frequency components experience a constant time delay. As M increases, the delay also increases, given by $(M - 1)/2$ samples. The small irregularities in the phase response in the stopband occur near frequencies where the magnitude is very close to zero.

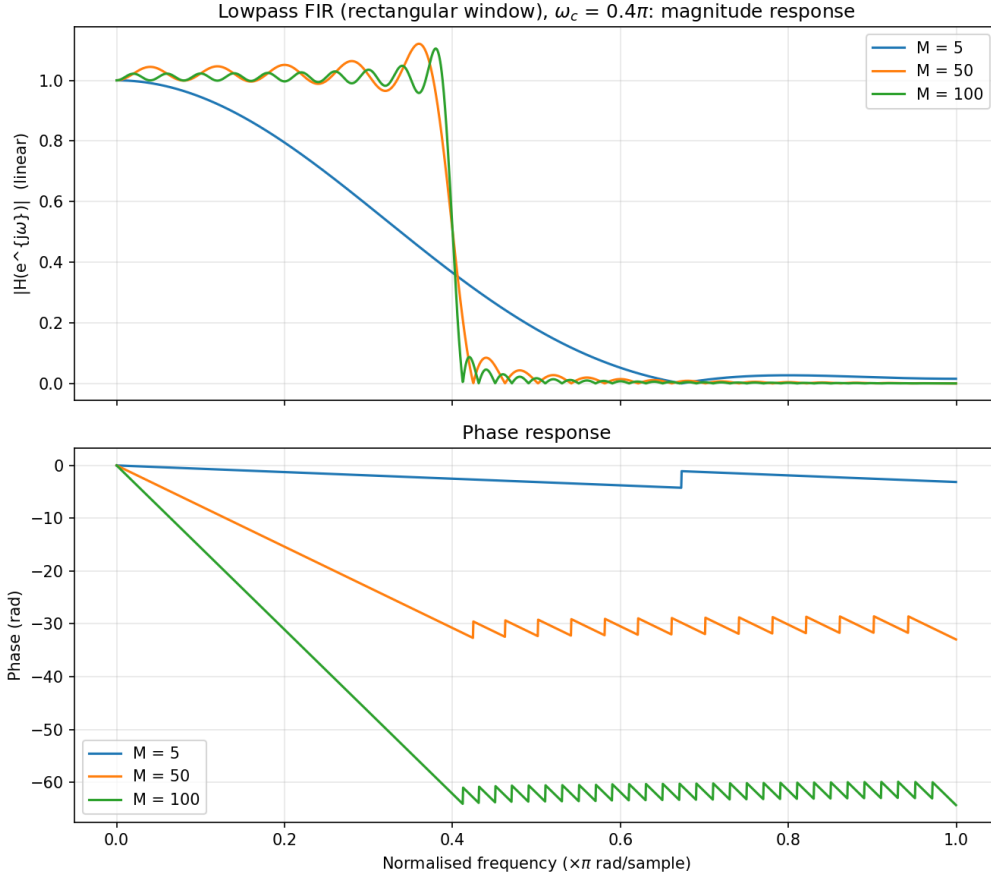


Figure 27: Lowpass FIR filter ($\omega_c = 0.4\pi$) built with a rectangular window: magnitude response (top) and phase response (bottom), for $M = 5, 50, 100$.

3.1.2 Comparison of Rectangular, Hanning, Hamming and Blackman Windows ($M = 50$)

Figure 28 shows the four windows in the time domain. The rectangular window has a constant value with sharp edges. The Hanning and Hamming windows have smooth raised-cosine shapes. The Hanning window reaches zero at the edges, while the Hamming window has a small non-zero value of approximately 0.08 at the edges. The Blackman window has a smoother shape and its values are close to zero at the edges.

The corresponding magnitude responses in Figure 29 show the trade-off between transition width and sidelobe attenuation. The rectangular window has the largest ripple and

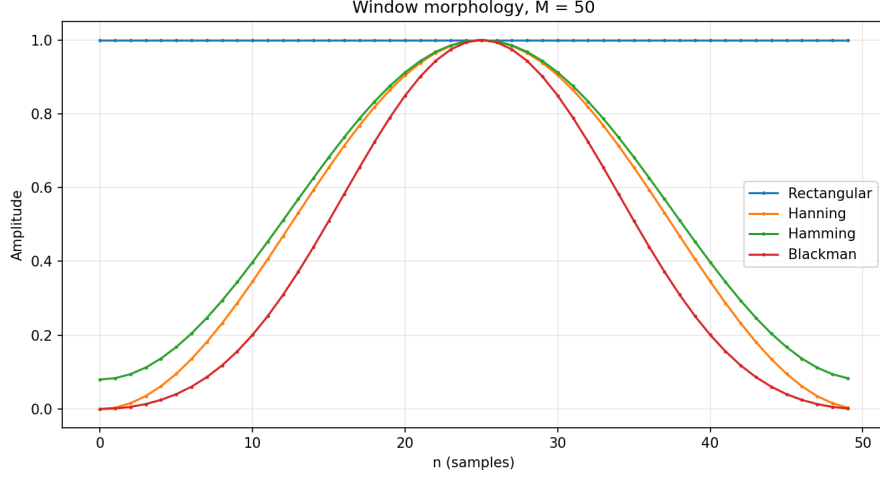


Figure 28: Window morphology for the rectangular, Hanning, Hamming and Blackman windows, $M = 50$.

the highest sidelobes, indicating greater spectral leakage. The Hanning and Hamming windows have lower sidelobes, while the Blackman window provides the greatest sidelobe attenuation. However, the smoother windows have wider main lobes and therefore wider transition bands than the rectangular window. This shows that reducing spectral leakage comes at the cost of reduced frequency resolution.

Figure 30 shows that all four windowed filters have a linear phase response in the pass-band. This is because all the windows are symmetric, which gives a constant group delay of

$$\frac{M - 1}{2} = 24.5$$

samples for each filter. The phase differences after the cutoff frequency are caused by the different frequency responses and zero locations of the filters.

3.2. FIR Filter Design and Application Using the Kaiser Window

3.2.1 Signal Inspection

The raw ECG with noise signal is shown in Figure 31. The signal contains the main ECG waveform along with low-frequency baseline variation and high-frequency noise.

The PSD of the noisy ECG is shown in Figure 32. Most of the ECG energy is concentrated in the low-frequency range, which is consistent with the typical bandwidth of a normal ECG. The peaks at 50, 100 and 150 Hz are due to powerline interference and its harmonics, while the higher-frequency components are mainly due to noise.

3.2.2 Filter Parameters and Kaiser Window Design

Based on the PSD observations above, three FIR filters were designed using the Kaiser window method via `scipy.signal.kaiserord()` and `scipy.signal.firwin()`:

- **Highpass filter:** stopband edge $f_{stop} = 3$ Hz, passband edge $f_{pass} = 5$ Hz and $\delta = 0.01$.

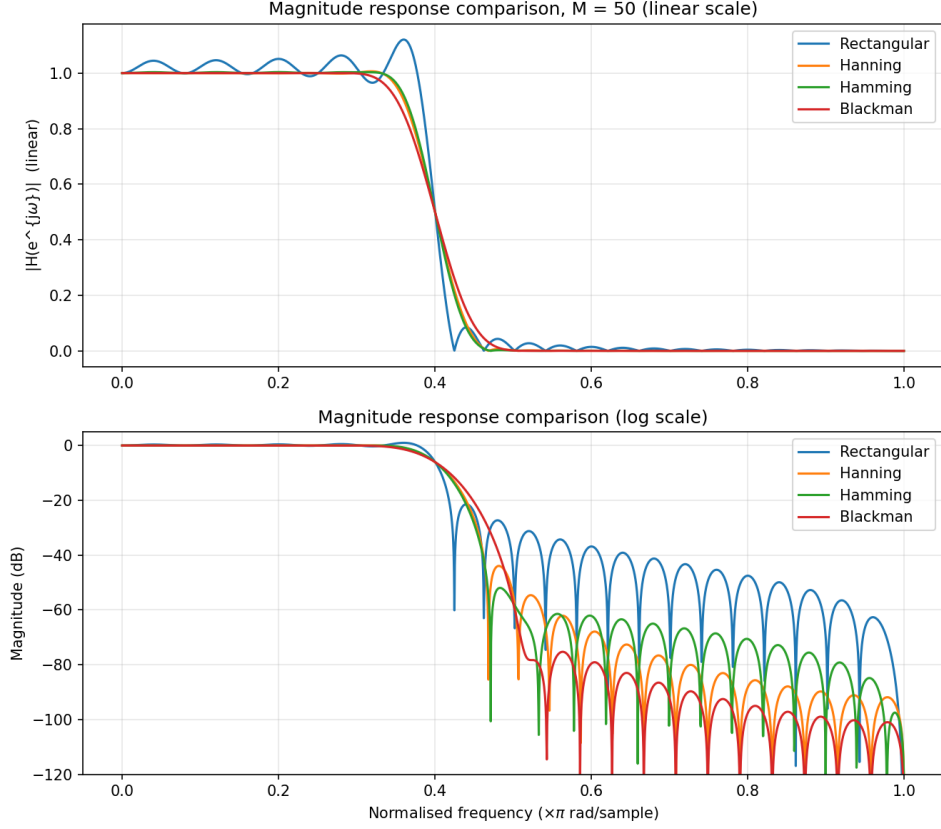


Figure 29: Magnitude response comparison for the four windows, $M = 50$: linear scale (top) and log (dB) scale (bottom).

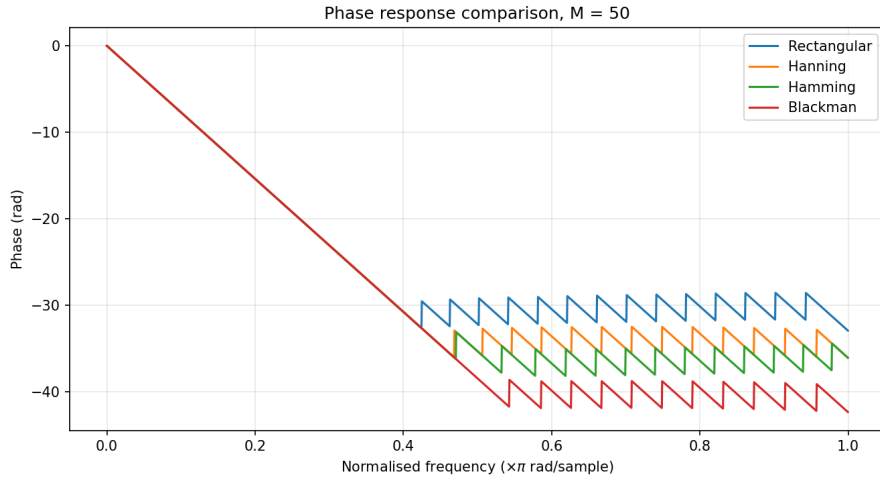


Figure 30: Phase response comparison for the four windows, $M = 50$.

- **Lowpass filter:** passband edge $f_{pass} = 125$ Hz, stopband edge $f_{stop} = 135$ Hz, and $\delta = 0.01$.
- **Notch/comb filter:** three narrow stopbands centred at $f_{stop1} = 50$ Hz, $f_{stop2} = 100$ Hz and $f_{stop3} = 150$ Hz (each ± 2 Hz wide).

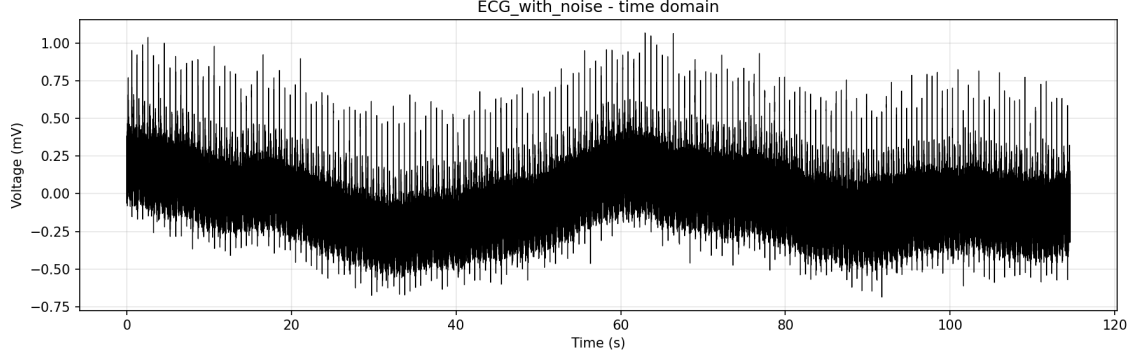


Figure 31: Time-domain ECG with noise recording.

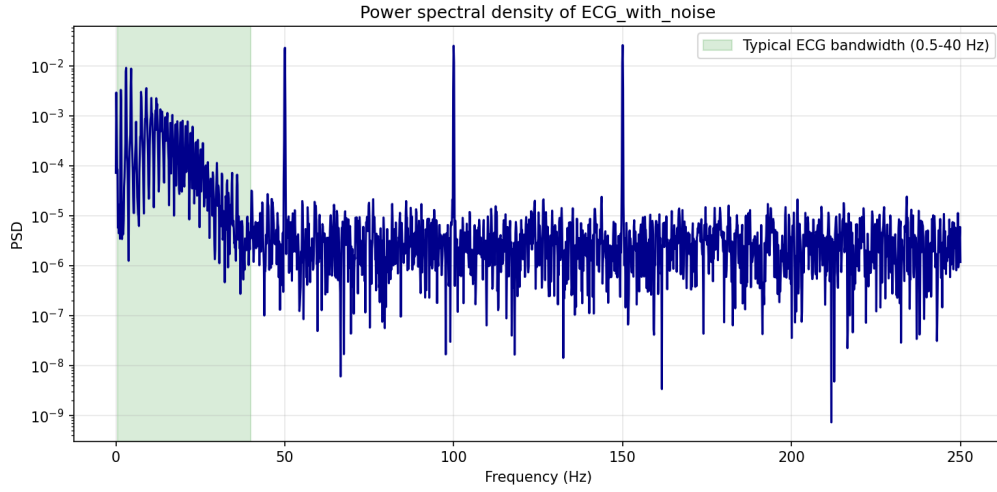


Figure 32: Power spectral density estimate of ECG with noise, with the typical diagnostic ECG bandwidth (0.5-40 Hz) shaded.

A common target attenuation of $A = 40$ dB was used for the filters. The Kaiser window parameters were calculated using

$$M \approx \frac{A - 8}{2.285 \Delta\omega}, \quad \Delta\omega = 2\pi \frac{\Delta f}{f_s}$$

and

$$\beta = \begin{cases} 0.1102(A - 8.7), & A > 50, \\ 0.5842(A - 21)^{0.4} + 0.07886(A - 21), & 21 \leq A \leq 50, \\ 0, & A < 21. \end{cases}$$

For $A = 40$ dB, this gives $\beta \approx 3.40$. The corresponding filter parameters obtained using `scipy.signal.kaiserord()` are:

Filter	Transition width	M	β
Highpass	2 Hz (3–5 Hz)	561	3.40
Lowpass	10 Hz (125–135 Hz)	113	3.40
Notch/comb	1 Hz (per band edge)	1119	3.40

3.2.3 Window and Response Visualisation

Figure 33 shows the Kaiser window, magnitude response, and phase response of the three filters.

The highpass filter strongly attenuates frequencies below approximately 5 Hz while allowing higher frequencies to pass. The lowpass filter allows frequencies up to approximately 125 Hz and attenuates higher frequencies. The notch filter produces three narrow rejection bands at 50, 100 and 150 Hz to remove the powerline interference.

The phase responses of all three filters are approximately linear, which indicates that they have a constant group delay.

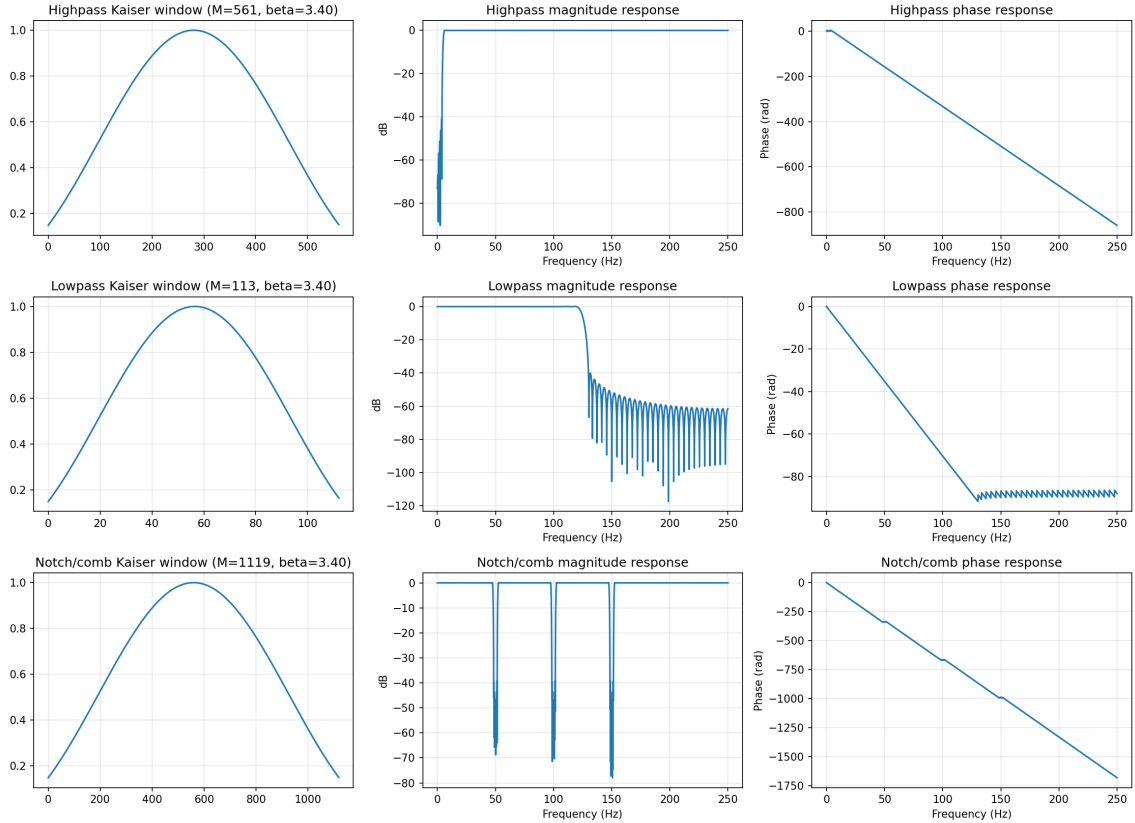


Figure 33: Kaiser window shape (left), magnitude response (middle) and phase response (right) for the highpass, lowpass, and notch filters.

3.2.4 Sequential Filtering with Group Delay Compensation

Each filter was applied sequentially using `scipy.signal.lfilter()`, with the constant group delay of $(M - 1)/2$ samples. Figure 34 shows the effect of each stage. The highpass filter removes the slow baseline wander and brings the signal closer to zero. The lowpass filter provides further smoothing by reducing high-frequency components. The final notch filter removes the powerline interference and makes the QRS-like peaks clearer.

3.2.5 Combined Response and Final PSD

The combined magnitude response of the cascaded filter is shown in Figure 35.

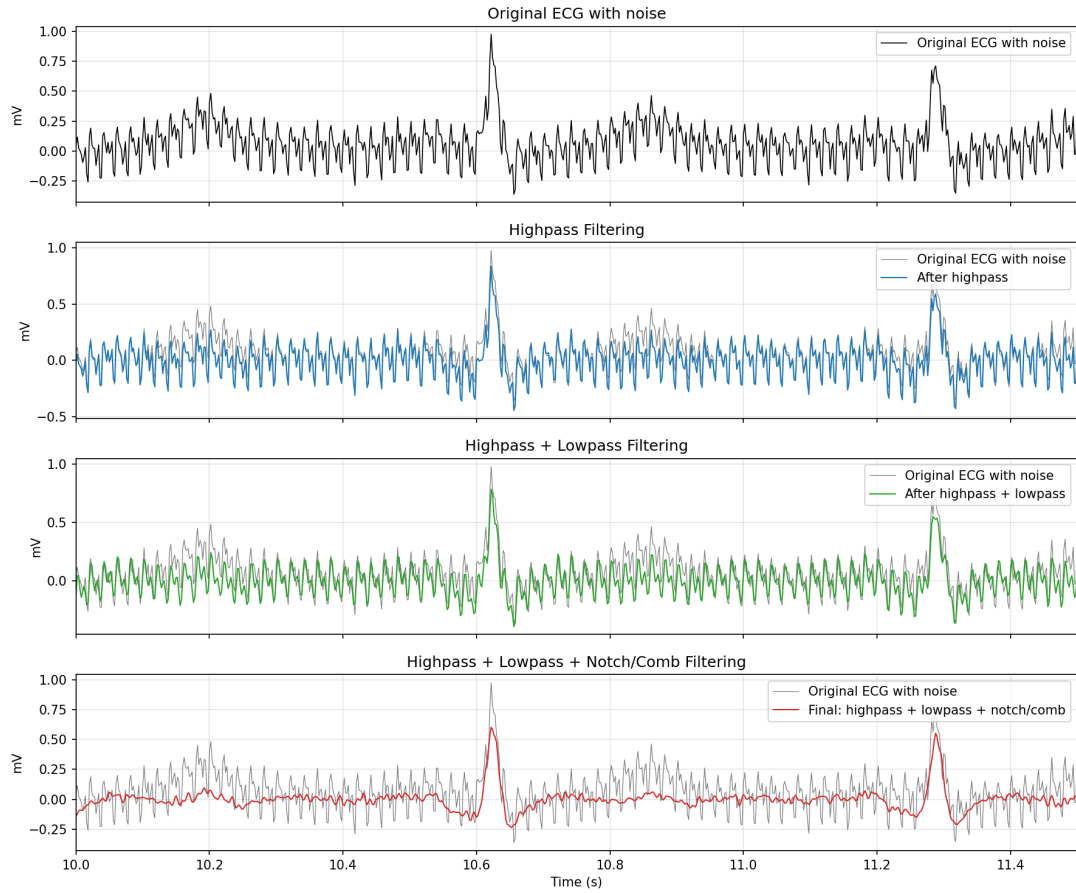


Figure 34: Time-domain effect of applying the highpass, lowpass and notch filters sequentially, with group delay compensation at each stage (Zoomed in for clear visualisation).

It clearly shows a composite passband between approximately 5 and 125 Hz, with three sharp notches at 50, 100 and 150 Hz and strong attenuation outside the 5–125 Hz band, exactly matching the intended design.

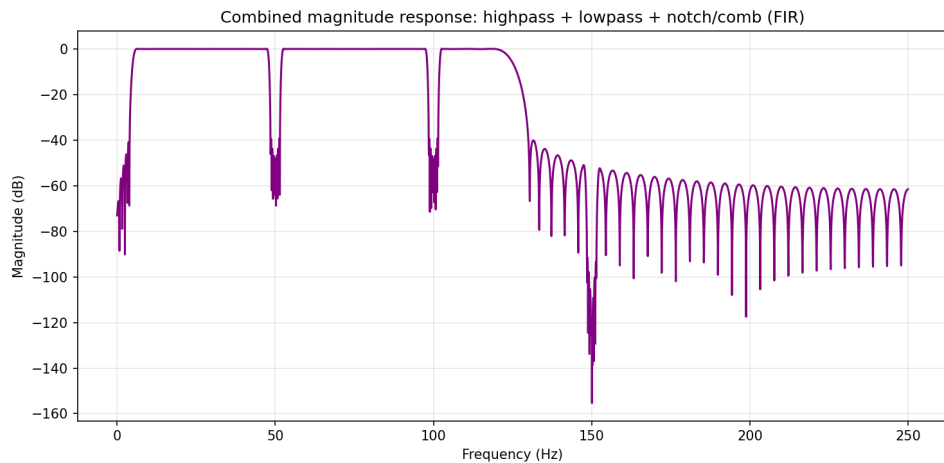


Figure 35: Combined magnitude response of the cascaded highpass, lowpass and notch FIR filters.

Figure 36 compares the PSD of the original noisy ECG against the final filtered signal.

The three powerline peaks at 50, 100 and 150 Hz are suppressed heavily in magnitude, confirming effective removal of the narrowband interference. The filter also reduces unwanted frequencies below 5 Hz and above 125 Hz while preserving the ECG frequencies in the 5–40 Hz range. The filter successfully removes the main noise while keeping the important ECG signal.

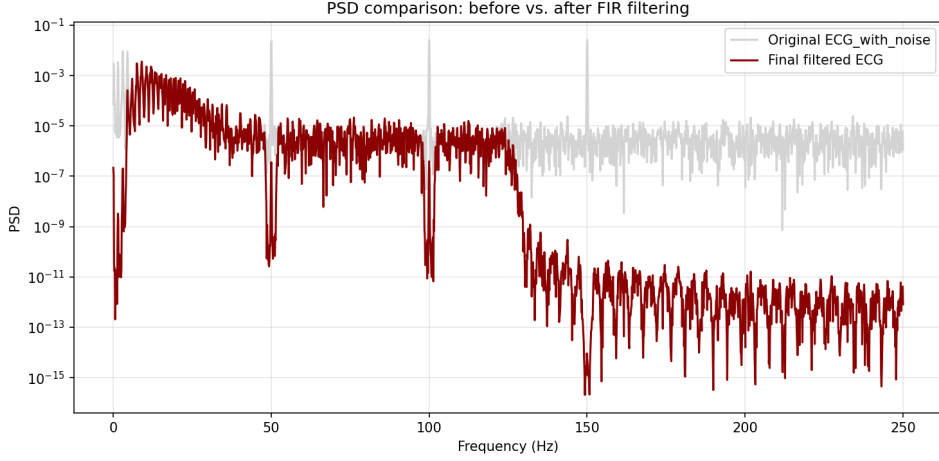


Figure 36: PSD comparison between the original `ECG_with_noise` signal and the final filtered ECG.

4. IIR Filters

4.1. Realizing IIR Filters

4.1.1 Butterworth Lowpass Filter

A 4th-order Butterworth lowpass filter was designed using `scipy.signal.butter()` with a cutoff frequency of 125 Hz, which is the same cutoff frequency used for the FIR lowpass filter in Section 3.2.

The filter was designed using

$$[b, a] = \text{butter}(4, 125, \text{btype}='low', \text{fs} = 500).$$

The resulting numerator coefficients were

$$b = [0.09398085, 0.37592341, 0.56388511, 0.37592341, 0.09398085]$$

and the denominator coefficients were

$$a = [1.00000000, -2.49800181 \times 10^{-16}, 0.486028822, -6.63518679 \times 10^{-17}, 0.0176648009].$$

It should be noted that the full FIR attenuation specification was not used for the IIR design. An initial attempt to match the FIR specifications using `scipy.signal.buttord()` resulted in a very high order of $N = 42$. This caused numerical problems when using the direct-form (b, a) coefficients. Therefore, a fixed order of $N = 4$ was used. This gives a simpler and numerically stable IIR filter for comparison.

Figure 37 shows the magnitude, phase and group delay responses. The Butterworth filter has a smooth magnitude response without ripple. The phase response is nonlinear,

and the group delay changes with frequency. The group delay increases around the cutoff frequency and then decreases at higher frequencies. This is different from the FIR filter, which has a constant group delay because it has a linear phase response.

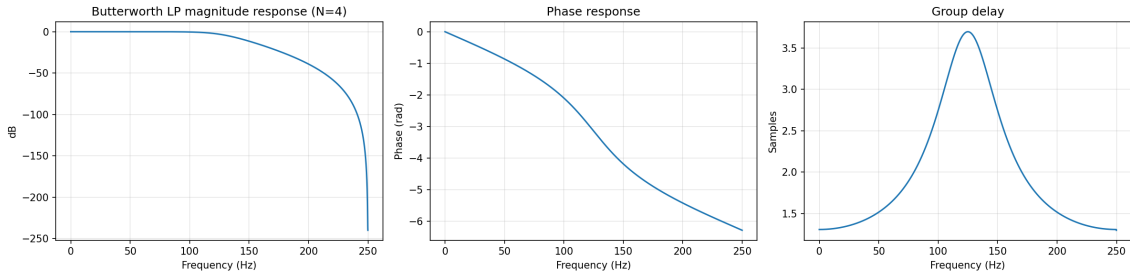


Figure 37: Butterworth lowpass filter ($N = 4$, cutoff 125 Hz): magnitude response (left), phase response (middle), and group delay (right).

4.1.2 Highpass and Comb Filters

Figure 38 shows the magnitude, phase and group delay responses of the Butterworth highpass filter and the IIR comb filter. The Butterworth highpass filter has order $N = 4$ and a cutoff frequency of 5 Hz. The filter coefficients obtained using `scipy.signal.butter()` are

$$b = [0.92117099, -3.68468397, 5.52702596, -3.68468397, 0.92117099]$$

and

$$a = [1.00000000, -3.83582554, 5.52081914, -3.53353522, 0.84855600].$$

The comb filter was designed using `scipy.signal.iircomb()` with a fundamental frequency of 50 Hz and $Q = 12.5$. The resulting coefficients are

$$b = [0.88783976, 0, 0, 0, 0, 0, 0, 0, 0, 0, -0.88783976]$$

and

$$a = [1.00000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, -0.77567951].$$

The comb filter removes the 50 Hz powerline interference and its harmonics. Unlike the FIR comb filter in Section 3.2, which was designed for 50, 100 and 150 Hz, the IIR comb filter automatically produces notches at the harmonics of 50 Hz up to the Nyquist frequency. Therefore, an additional notch can be seen at 200 Hz.

The highpass filter shows a sharp feature in the group delay near 0 Hz. The comb filter also shows changes in group delay around its notch frequencies because the magnitude response becomes very small at these frequencies.

4.1.3 Combined Response: FIR vs. IIR

Figure 39 compares the combined magnitude responses of the FIR and IIR filters. The FIR filter has deeper notches at 50, 100 and 150 Hz, while the IIR comb filter has shallower notches. This is because the FIR filter uses many taps to achieve higher attenuation, while the IIR filter uses a fixed order of 4 and a low-order comb filter. The IIR lowpass

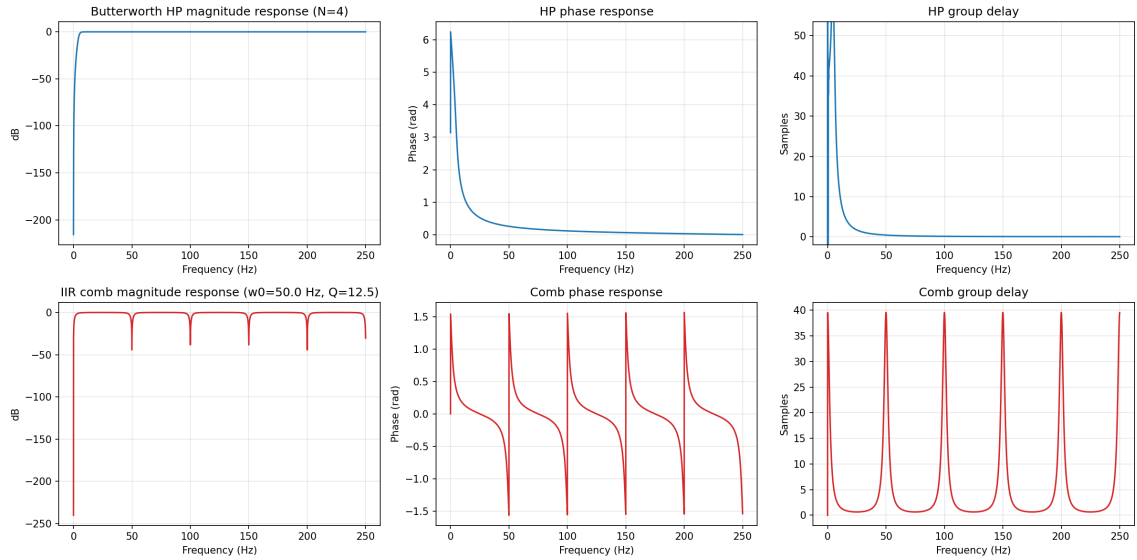


Figure 38: Butterworth highpass filter (top row) and IIR comb filter (bottom row): magnitude, phase and group delay responses.

response decreases smoothly after 125 Hz, while the FIR response has more oscillations in the stopband.

Overall, the FIR filter provides sharper and deeper rejection but requires many more coefficients. The IIR filter achieves a similar overall filtering shape with a much lower order, but has a nonlinear phase response, which can introduce more waveform distortion when used in a single forward pass.

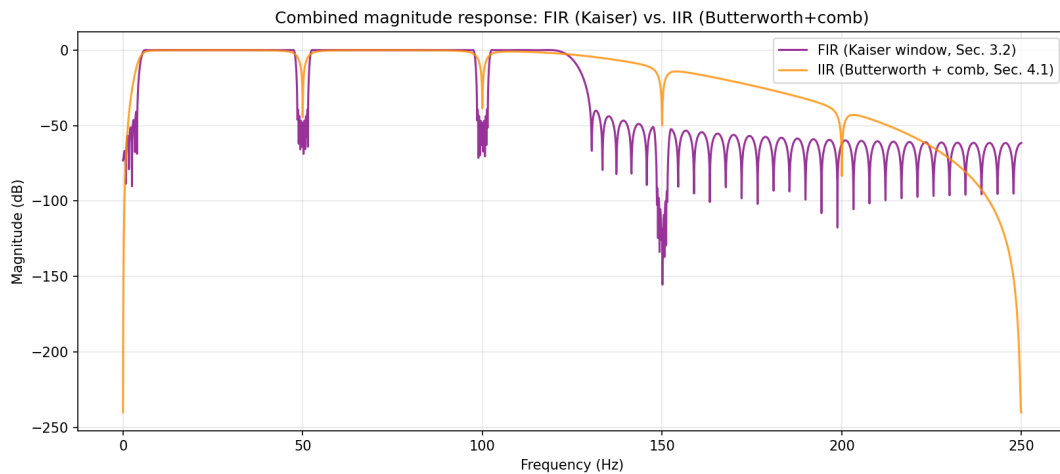


Figure 39: Combined magnitude response: FIR (Kaiser window, Section 3.2) vs. cascaded IIR (Butterworth highpass + lowpass + comb, Section 4.1).

4.2. Filtering Methods for IIR Filters

4.2.1 Forward vs. Forward-Backward Filtering: Time Domain

Figure 40 compares two ECG beats after FIR filtering, forward IIR filtering using `lfilter()`, and forward-backward IIR filtering using `filtfilt()`.

The FIR-filtered signal and the forward-backward IIR signal are visually very similar. In contrast, the forward-only IIR signal shows a time shift and a different QRS shape. This occurs because the IIR filter has a nonlinear phase response and a frequency-dependent group delay. Different frequency components of the QRS complex are therefore delayed by different amounts.

The `filtfilt()` method filters the signal in the forward direction and then in the reverse direction. The phase shift introduced by the first pass is approximately cancelled by the reverse pass, giving a near-zero-phase result. Therefore, the forward-backward IIR signal does not have the same time shift seen in the forward-only result.

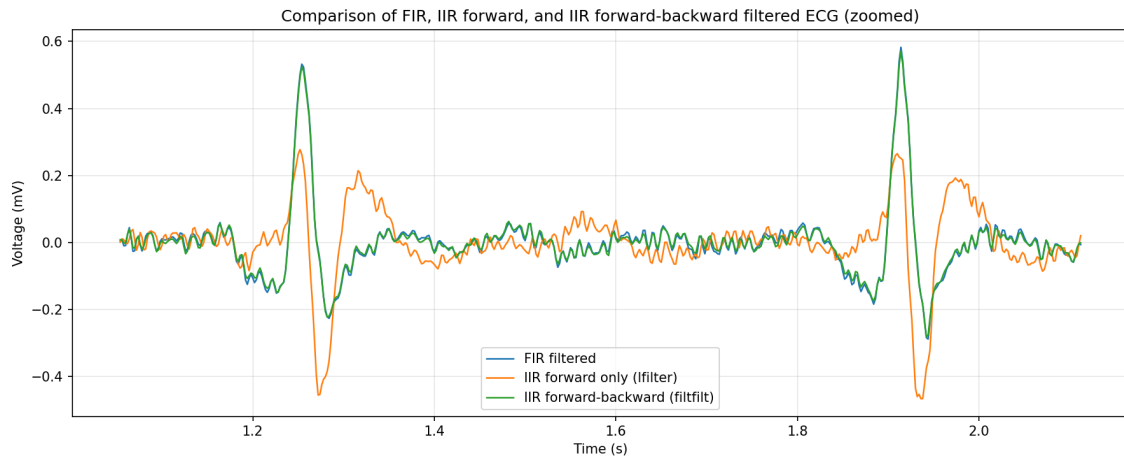


Figure 40: Zoomed time-domain comparison of FIR-filtered, IIR forward-only (`lfilter`), and IIR forward-backward (`filtfilt`) filtered ECG.

4.2.2 Forward vs. Forward-Backward Filtering: PSD

Figure 41 compares the PSDs of the FIR-filtered, forward IIR-filtered, and forward-backward IIR-filtered signals.

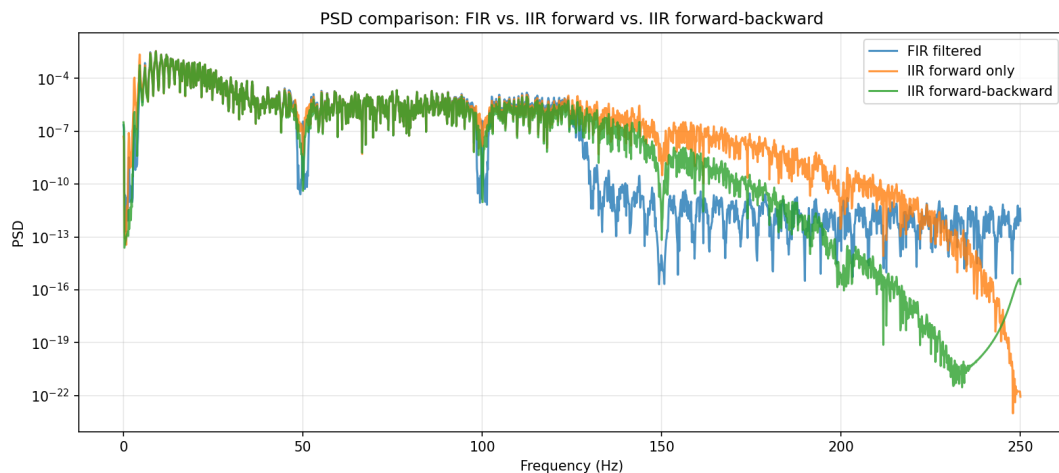


Figure 41: PSD comparison: FIR-filtered, IIR forward-only, and IIR forward-backward filtered ECG.

All three signals have a similar passband between approximately 5 and 125 Hz, with attenuation around the powerline frequencies. Above approximately 130 Hz, the forward-only IIR signal has more remaining high-frequency energy than the other two signals. The forward-backward IIR result has stronger attenuation in the stopband. This is because `filtfilt()` applies the filter twice. As a result, the forward-backward IIR filter can provide stronger stopband attenuation than the single-pass IIR filter.

Overall, the time-domain results show the effect of the IIR nonlinear phase, while the PSD results show the difference in the magnitude response and attenuation of the filtering methods.